

RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Sfax

Institut Supérieur d'Informatique et de Multimédia de Sfax

Département Informatique et Multimédia

RAPPORT DE STAGE D'ÉTÉ

Présenté dans le cadre du cycle d'ingénieur en technologies web et multimédia

Sujet

CFSCMS

**Conception et réalisation d'une plateforme web et mobile de gestion des
hébergements étudiants**

Application web (administration) et application mobile Android (étudiants) — Flutter & Firebase

Réalisé par : **Naziha Goubaa**

Encadré par : **Mabrouk Abaab**

Organisme d'accueil

Centre Sectoriel de Formation en Construction Métallique et Soudure — Médenine

Année universitaire 2025 / 2026

Remerciements

Au terme de ce stage d'été, je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail.

Je remercie tout d'abord mon encadrant, Monsieur Mabrouk Abaab, pour ses conseils avisés, sa disponibilité et son accompagnement tout au long de la réalisation de ce projet. Ses orientations méthodologiques et ses remarques ont été précieuses pour structurer le travail, consolider la démarche d'analyse et améliorer la qualité de ses différentes parties.

Mes remerciements s'adressent également à la direction et à l'ensemble du personnel du Centre Sectoriel de Formation en Construction Métallique et Soudure de Médenine, pour leur accueil, leur confiance et la disponibilité dont ils ont fait preuve tout au long du stage. Les informations qu'ils m'ont fournies sur le fonctionnement réel du centre, qu'il s'agisse de la gestion des hébergements, du suivi des paiements ou de l'organisation des formations, ont été déterminantes pour concevoir une plateforme fidèle aux besoins effectifs du terrain.

Je souhaite enfin remercier vivement l'ensemble des enseignants et des cadres de l'Institut Supérieur d'Informatique et de Multimédia de Sfax pour la qualité de la formation qu'ils m'ont dispensée durant ce cursus, ainsi que les membres du jury pour l'intérêt qu'ils ont porté à ce travail et pour avoir accepté de l'évaluer.

Table des matières

Introduction générale	1
Chapitre 1 — Cadre général du projet.....	3
1.1 Introduction	3
1.2 Présentation de l'organisme d'accueil	3
1.3 Étude de l'existant	4
1.3.1 La gestion manuelle (papier)	4
1.3.2 Les tableurs (Excel).....	4
1.3.3 Les progiciels de gestion intégrés génériques (exemple : Odoo).....	4
1.3.4 Les plateformes de gestion universitaire en ligne	4
1.3.5 Critique de l'existant	5
1.4 Problématique	5
1.5 Solution proposée	6
1.6 Objectifs du projet	6
1.7 Identification des acteurs.....	7
1.8 Conclusion.....	7
Chapitre 2 — Analyse et spécification des besoins	8
2.1 Introduction	8
2.2 Besoins fonctionnels par module.....	8
2.3 Règles de gestion identifiées	9
2.4 Besoins non fonctionnels	10
2.5 Conclusion.....	10
Chapitre 3 — Conception de la solution	11
3.1 Introduction	11
3.2 Diagramme de cas d'utilisation.....	11
3.3 Description des cas d'utilisation	12
3.4 Scénarios détaillés de cas d'utilisation.....	13
3.5 Diagramme de classes.....	15

3.6 Vues dynamiques : les diagrammes de séquence	17
3.6.1 Diagramme de séquence « S'authentifier »	17
3.6.2 Diagramme de séquence « Soumettre une demande d'hébergement »	18
3.6.3 Diagramme de séquence « Traiter une demande d'hébergement »	19
3.7 Architecture logicielle globale	20
3.8 Architecture générale et environnement de déploiement	21
3.9 Modèle de données : les collections Firestore	22
3.10 Conception de la logique métier critique	24
3.10.1 Éligibilité à l'hébergement	24
3.10.2 Inscription à une formation	24
3.10.3 Approbation d'une demande d'hébergement	24
3.10.4 Gestion de l'occupation des chambres	25
3.11 Charte graphique et composants d'interface	25
3.12 Conclusion	25
Chapitre 4 — Environnement de travail et choix technologiques	26
4.1 Introduction	26
4.2 Environnement matériel	26
4.3 Environnement logiciel	26
4.4 Choix technologiques et justification	27
4.4.1 Flutter	27
4.4.2 Firebase comme backend	27
4.4.3 Provider pour la gestion d'état	27
4.5 Bibliothèques et dépendances principales	28
4.6 Conclusion	28
Chapitre 5 — Réalisation	29
5.1 Introduction	29
5.2 Organisation du code source	29
5.3 Authentification et gestion des rôles	29
5.4 Espace administrateur (interface web)	29
5.5 Espace étudiant (application mobile)	36
5.6 Génération de documents PDF	40

5.7 Sécurité des données	40
5.8 Conclusion.....	40
Conclusion générale et perspectives	41
Bilan du projet.....	41
Perspectives d'évolution	42
Nétographie.....	43

Liste des figures

Figure 1 : Diagramme de cas d'utilisation global du système	12
Figure 2 : Diagramme de classes de l'application CFSCMS	16
Figure 3 : Diagramme de séquence « S'authentifier »	18
Figure 4 : Diagramme de séquence « Soumettre une demande d'hébergement »	19
Figure 5 : Diagramme de séquence « Traiter une demande d'hébergement »	20
Figure 6 : Architecture logicielle en couches de l'application CFSCMS	21
Figure 7 : Architecture générale et environnement de déploiement	22
Figure 8 : Interface d'authentification de l'administrateur	30
Figure 9 : Tableau de bord administrateur	30
Figure 10 : Interface de gestion des étudiants	31
Figure 11 : Formulaire de création d'un compte étudiant	32
Figure 12 : Interface de gestion des formations	32
Figure 13 : Interface des demandes d'hébergement et panneau d'éligibilité	33
Figure 14 : Approbation d'une demande et attribution d'une chambre	34
Figure 15 : Interface de gestion des chambres (blocs A et B)	34
Figure 16 : Interface de gestion des paiements et reçu PDF	35
Figure 17 : Interface d'envoi des notifications	36
Figure 18 : Interface de connexion de l'application mobile étudiant	36
Figure 19 : Tableau de bord personnel de l'étudiant	37
Figure 20 : Interface « Mon profil »	37
Figure 21 : Inscription à une formation côté étudiant	38
Figure 22 : Chambre attribuée et section « Mes binômes »	39
Figure 23 : Historique des paiements et génération du reçu PDF	39

Liste des tableaux

Tableau 1 : Étude comparative des solutions existantes	5
Tableau 2 : Identification des acteurs du système.....	7
Tableau 3 : Besoins fonctionnels par module	8
Tableau 4 : Cas d'utilisation — Administrateur	12
Tableau 5 : Cas d'utilisation — Étudiant.....	13
Tableau 6 : Scénario du cas d'utilisation « S'inscrire à la formation »	13
Tableau 7 : Scénario du cas d'utilisation « Gérer les hébergements (Modifier) »	14
Tableau 8 : Attributs et méthodes notables des classes du modèle.....	16
Tableau 9 : Description des collections Firestore.....	23
Tableau 10 : Environnement matériel.....	26
Tableau 11 : Logiciels utilisés	26
Tableau 12 : Bibliothèques et dépendances principales.....	28

Introduction générale

La transformation numérique des établissements de formation professionnelle est devenue un enjeu majeur en Tunisie. Le secteur joue en effet un rôle central dans l'insertion des jeunes sur le marché de l'emploi, en particulier dans des filières techniques telles que la construction métallique et la soudure. Les centres de formation professionnelle gèrent pourtant, au quotidien, un volume important d'informations administratives : dossiers des étudiants, inscriptions aux formations, demandes d'hébergement, attribution des chambres, suivi des paiements et communication avec les apprenants. Lorsque cette gestion repose sur des procédures manuelles, des registres papier ou des tableurs isolés, elle devient rapidement source de lenteurs, d'erreurs de saisie, de pertes d'information et d'un manque de visibilité globale pour la direction.

Le Centre Sectoriel de Formation en Construction Métallique et Soudure de Médenine (CFSCMS) illustre bien cette réalité. Rattaché au réseau de l'Agence Tunisienne de la Formation Professionnelle (ATFP), il accueille chaque année des stagiaires venant de différents gouvernorats du pays, parfois éloignés du site de formation. Une part importante d'entre eux, résidant à plus de trente kilomètres du centre, sollicite un hébergement au sein du foyer afin de garantir leur assiduité et de réduire les contraintes de déplacement quotidien. Le traitement de ces demandes, la vérification des critères d'éligibilité, l'attribution des chambres, le suivi des paiements d'inscription, d'hébergement et de mensualités, ainsi que la délivrance des reçus, constituent une charge administrative considérable lorsqu'ils sont effectués manuellement ou au moyen d'outils bureautiques disparates.

C'est dans ce contexte que s'inscrit le présent stage d'été, qui consiste à concevoir et à réaliser « CFSCMS », une plateforme numérique multiplateforme développée avec Flutter et Firebase, composée d'une interface web destinée à l'administration et d'une application mobile Android destinée aux étudiants. Les deux interfaces s'appuient sur une base de données commune et synchronisée en temps réel, ce qui permet de centraliser, sécuriser et automatiser l'ensemble du processus de gestion des hébergements du centre, depuis la création du compte de l'étudiant jusqu'au suivi financier, en passant par l'inscription aux formations, la vérification automatique de l'éligibilité, l'attribution des chambres et l'envoi de notifications.

Le présent rapport est organisé en cinq chapitres. Le premier chapitre présente le cadre général du projet : l'organisme d'accueil, l'étude critique de l'existant, la problématique et la

solution proposée. Le deuxième chapitre est consacré à l'analyse et à la spécification des besoins fonctionnels et non fonctionnels. Le troisième chapitre détaille la conception de la solution : modélisation des cas d'utilisation, architecture logicielle, modèle de données, diagramme de classes, diagrammes de séquence et logique métier critique. Le quatrième chapitre justifie les choix technologiques et présente l'environnement de travail. Enfin, le cinquième chapitre décrit la réalisation concrète des différents modules de l'application, à travers les interfaces développées côté administrateur et côté étudiant. Le rapport se termine par une conclusion générale qui dresse le bilan du projet et ouvre sur des perspectives d'évolution.

Chapitre 1 — Cadre général du projet

1.1 Introduction

Ce chapitre est consacré à la présentation du cadre général dans lequel s'est déroulé le projet. Nous présentons d'abord l'organisme d'accueil, puis nous menons une étude critique de l'existant afin d'en dégager les insuffisances. Nous exposons ensuite la problématique traitée, la solution proposée ainsi que les objectifs poursuivis, avant d'identifier les acteurs appelés à interagir avec le système.

1.2 Présentation de l'organisme d'accueil

Le Centre Sectoriel de Formation en Construction Métallique et Soudure de Médenine (CFSCMS) est un établissement public de formation professionnelle relevant de l'Agence Tunisienne de la Formation Professionnelle (ATFP). Implanté à El Feja, dans le gouvernorat de Médenine, il a été mis en place dans le cadre d'un programme de coopération franco-tunisien de développement de la formation professionnelle, aux côtés d'un nombre restreint de centres de ce type à l'échelle nationale. Le centre est spécialisé dans les métiers de la construction métallique, de la chaudronnerie et de la soudure, et a pour mission de former des techniciens et des ouvriers qualifiés répondant aux besoins du tissu industriel régional et national, à travers des cursus théoriques et des périodes de stage en entreprise.

Le centre accueille des stagiaires provenant de différents gouvernorats du pays, dont une partie doit être hébergée sur place pendant la durée de la formation. Il met à leur disposition un foyer d'hébergement organisé en deux blocs (bloc A et bloc B), avec des formules adaptées : pension complète, demi-pension ou régime externe. Outre sa mission pédagogique, le centre assure ainsi une mission administrative importante, qui relève d'un service dédié en lien avec la direction pédagogique gérant les formations : la gestion des dossiers des étudiants, l'encaissement des frais d'inscription et d'hébergement, l'attribution des chambres et la communication avec les apprenants. C'est cette dimension administrative qui constitue le périmètre du présent projet.

1.3 Étude de l'existant

L'étude de l'existant permet d'analyser les modes de gestion actuels ainsi que les solutions disponibles sur le marché, afin d'en identifier les insuffisances et de spécifier les grands axes de notre travail.

1.3.1 La gestion manuelle (papier)

C'est le mode de gestion historique du centre : formulaires papier pour les demandes d'hébergement, registres pour le suivi de l'occupation des chambres et carnets à souche pour les reçus de paiement. Cette organisation est simple à mettre en œuvre, mais elle est lente, sujette aux erreurs et aux pertes de documents, et ne fournit aucune statistique en temps réel à la direction.

1.3.2 Les tableurs (Excel)

Certains services utilisent des feuilles de calcul pour suivre les étudiants et les paiements. Les tableurs améliorent la lisibilité par rapport au papier, mais restent des fichiers isolés, mono-utilisateur, sans contrôle d'accès, sans notification et sans interface dédiée à l'étudiant. La cohérence des données dépend entièrement de la rigueur de la personne qui saisit, et le risque de divergence entre plusieurs copies d'un même fichier est élevé.

1.3.3 Les progiciels de gestion intégrés génériques (exemple : Odoo)

Les progiciels de gestion intégrés open source, à l'image d'Odoo, proposent des modules de gestion de la scolarité et de l'internat. Ils sont puissants, mais complexes à déployer : ils nécessitent un serveur dédié et un paramétrage lourd, et leurs modules génériques ne couvrent pas les règles métier spécifiques du centre, telles que le critère de distance d'éligibilité, les formules de pension ou la génération automatique des matricules.

1.3.4 Les plateformes de gestion universitaire en ligne

Il existe également des plateformes web de gestion des foyers universitaires. Elles sont généralement conçues pour de grandes institutions, proposées sous forme d'abonnements coûteux, rarement disponibles en version mobile pour l'étudiant, et difficilement adaptables au contexte particulier d'un centre de formation sectoriel de taille plus modeste.

1.3.5 Critique de l'existant

Le tableau 1 résume les insuffisances des solutions étudiées au regard des fonctionnalités que notre plateforme doit assurer.

Tableau 1 : Étude comparative des solutions existantes

Critère	Gestion papier	Tableur Excel	ERP générique	Plateforme proposée
Accès temps réel multi-utilisateurs	Non	Non	Oui	Oui
Espace mobile dédié à l'étudiant	Non	Non	Non	Oui
Vérification automatique de l'éligibilité (distance, paiement)	Non	Non	Non	Oui
Gestion de l'occupation des chambres en temps réel	Non	Partielle	Oui	Oui
Génération automatique de reçus et attestations PDF	Non	Non	Partielle	Oui
Notifications aux étudiants	Non	Non	Partielle	Oui
Coût et complexité de déploiement	Faible	Faible	Élevé	Faible

1.4 Problématique

L'analyse qui précède fait ressortir la problématique suivante : comment offrir à l'administration du CFSCMS un outil unique, fiable et accessible en temps réel pour gérer l'ensemble du cycle de vie d'une demande d'hébergement (soumission, instruction, attribution de chambre, facturation), tout en offrant au stagiaire une visibilité autonome sur sa demande et ses paiements, sans dépendre d'échanges papier ou d'appels téléphoniques répétés ?

Plus précisément, la solution recherchée doit résoudre les difficultés suivantes : la lenteur du traitement des demandes d'hébergement, l'absence de vérification systématique des critères d'éligibilité, le risque de sur-occupation ou de mauvaise affectation des chambres, le suivi

approximatif des paiements, et l'absence de canal de communication direct entre l'administration et les étudiants.

1.5 Solution proposée

Pour répondre à ces limites, ce projet propose la conception d'une plateforme intégrée baptisée « CFSCMS », composée de deux interfaces complémentaires s'appuyant sur une base de données commune et synchronisée en temps réel :

- une interface web d'administration, destinée au personnel du centre, offrant un tableau de bord temps réel et la gestion complète des étudiants, des formations, des chambres, des hébergements, des paiements et des notifications ;
- une application mobile Android destinée aux stagiaires, leur permettant de gérer leur profil, de s'inscrire à une formation, de soumettre une demande d'hébergement, de suivre son statut, de consulter leur chambre et leurs binômes, et de suivre leurs paiements.

Les deux interfaces partagent le même code source Flutter et la même base de données Firebase, ce qui garantit une cohérence totale des informations affichées de part et d'autre, ainsi qu'une mise à jour instantanée dès qu'une action est effectuée côté administration.

1.6 Objectifs du projet

- Concevoir un système d'information cohérent couvrant les étudiants, les formations, les hébergements, les chambres, les paiements et les notifications.
- Développer une application unique multiplateforme (web pour l'administration, mobile Android pour les stagiaires) à partir d'une base de code commune, réduisant les coûts et délais de développement.
- Centraliser toutes les données du centre dans une base unique, sécurisée et synchronisée en temps réel (Cloud Firestore).
- Automatiser les règles métier sensibles : éligibilité à l'hébergement, calcul et génération des échéances de paiement, synchronisation des compteurs d'occupation des chambres.
- Fiabiliser l'attribution des chambres grâce à des transactions atomiques mettant à jour simultanément la demande, la chambre et le statut de l'étudiant, même en cas d'accès concurrents.

- Sécuriser l'accès aux données selon le rôle de l'utilisateur (administrateur ou étudiant).
- Dématérialiser les documents administratifs : fiches étudiants, reçus de paiement et attestations, générés au format PDF.
- Améliorer la communication entre l'administration et les étudiants grâce à des notifications transmises en temps réel.

1.7 Identification des acteurs

Le système fait intervenir deux acteurs principaux, dont le tableau 2 précise le rôle.

Tableau 2 : Identification des acteurs du système

Acteur	Description	Rôle principal
Administrateur	Personnel administratif du centre, responsable de la gestion de l'hébergement et de la scolarité.	Gère les étudiants, les formations, les chambres, les hébergements, les paiements et les notifications ; consulte le tableau de bord.
Étudiant (stagiaire)	Apprenant inscrit à une formation du centre, éligible ou non à l'hébergement.	Gère son profil, s'inscrit à une formation, soumet une demande d'hébergement, consulte ses paiements et ses notifications.

1.8 Conclusion

Ce chapitre a permis de présenter l'organisme d'accueil ainsi que le cadre et le contexte du projet. L'étude critique de l'existant a mis en évidence les limites des approches manuelles, bureaucratiques et génériques, ce qui nous a permis de dégager la problématique et de définir les objectifs de la solution proposée. Le chapitre suivant est consacré à l'analyse et à la spécification des besoins de la plateforme.

Chapitre 2 — Analyse et spécification des besoins

2.1 Introduction

La phase d'analyse et de spécification des besoins constitue une étape fondamentale de tout projet informatique : elle permet de traduire les attentes des utilisateurs en un ensemble d'exigences claires et vérifiables, qui guideront la conception puis l'implémentation. Ce chapitre présente les besoins fonctionnels par module, les règles de gestion identifiées ainsi que les besoins non fonctionnels retenus pour la plateforme. La modélisation des cas d'utilisation, qui prolonge directement cette analyse, est présentée au chapitre suivant aux côtés des autres diagrammes de conception.

2.2 Besoins fonctionnels par module

Le tableau 3 synthétise, module par module, les fonctionnalités principales attendues de la plateforme.

Tableau 3 : Besoins fonctionnels par module

Module	Fonctionnalités principales
Authentification	Connexion sécurisée par e-mail / mot de passe, réinitialisation du mot de passe, redirection automatique selon le rôle (administrateur ou étudiant).
Tableau de bord	Statistiques en temps réel : nombre d'étudiants, demandes d'hébergement en attente, paiements en attente, chambres disponibles, répartition des étudiants par statut.
Gestion des étudiants	Recherche, ajout, modification, suppression, génération automatique du matricule, génération et impression de la fiche étudiant, envoi des identifiants par e-mail.
Gestion des formations	Ouverture de l'accès aux demandes, création de formations, définition des tarifs (inscription, hébergement résident, hébergement semi-résident), suivi de la capacité et des inscrits.
Gestion des hébergements	Ajout, modification et suppression des demandes d'hébergement, vérification automatique de l'éligibilité, affectation à une chambre.

Module	Fonctionnalités principales
Gestion des états	Attribution et mise à jour de l'état de l'étudiant (Résident, Semi-résident), en cohérence avec l'hébergement approuvé.
Gestion des chambres	Répartition en blocs (A / B), suivi de la capacité et des places occupées en temps réel, transfert ou échange d'étudiants entre chambres.
Gestion des paiements	Consultation des paiements, blocage ou élimination, changement de statut (en attente / payé) ; export du reçu officiel en PDF.
Notifications	Envoi de notifications ciblées ou générales aux étudiants, avec possibilité de partage par e-mail.
Espace étudiant	Suivi du statut de la demande d'hébergement, consultation des paiements et de la formation, gestion du profil et de la photo.

2.3 Règles de gestion identifiées

L'analyse du domaine métier a permis de dégager plusieurs règles de gestion structurantes, qui conditionnent directement la conception du système :

- Un étudiant n'est éligible à une demande d'hébergement que s'il n'est pas déjà résident ou semi-résident (statut « Non inscrit », « Inscrit » ou « Externe ») et s'il est rattaché à une formation.
- Un critère de distance minimale de trente kilomètres entre le domicile de l'étudiant et le centre, déterminée à partir d'une table de distances routières intégrée, est pris en compte comme facteur d'éligibilité à l'hébergement.
- L'inscription d'un étudiant à une formation payante déclenche automatiquement la création d'un paiement d'inscription au statut « En attente ».
- L'approbation d'une demande d'hébergement déclenche automatiquement la génération des échéances de paiement mensuelles correspondant à la durée de l'hébergement et au type choisi (pension complète ou demi-pension).
- Le statut de l'étudiant (Résident, Semi-résident, Externe) découle du type d'hébergement approuvé et détermine le tarif mensuel appliqué.

- La capacité d'une chambre ne peut jamais être dépassée : toute tentative d'affectation au-delà de la capacité disponible est rejetée par le système, de même que l'affectation à une chambre dont le genre autorisé ne correspond pas à celui de l'étudiant.

2.4 Besoins non fonctionnels

- Temps réel : toute modification effectuée côté administration (statut d'une demande, disponibilité d'une chambre, nouveau paiement, notification) doit être immédiatement visible côté étudiant, sans rechargement manuel, grâce aux flux (streams) Firestore.
- Cohérence des données : les opérations touchant plusieurs entités liées, comme l'approbation d'un hébergement qui modifie à la fois la demande, la chambre et l'étudiant, doivent être exécutées de manière atomique, afin d'éviter tout état incohérent en cas d'échec partiel ou d'accès concurrent.
- Sécurité : l'accès aux données doit être strictement contrôlé selon le rôle de l'utilisateur authentifié ; un étudiant ne doit jamais pouvoir consulter ou modifier les données d'un autre étudiant.
- Portabilité : l'application doit fonctionner aussi bien dans un navigateur web, pour le poste de l'administration, que sur un terminal Android, pour les stagiaires, à partir d'une base de code unique.
- Ergonomie : l'interface doit être simple d'utilisation pour un personnel non technique et pour des stagiaires peu familiers des outils numériques, avec une identité visuelle homogène entre les deux plateformes.
- Disponibilité et extensibilité : le service doit être accessible à tout moment via une infrastructure hébergée (Firebase), sans nécessiter de serveur maintenu par le centre, et doit pouvoir absorber une montée en charge progressive.

2.5 Conclusion

Ce chapitre a permis de spécifier les besoins fonctionnels et non fonctionnels de la plateforme ainsi que les règles de gestion structurantes qui en découlent. Cette étude constitue le socle de la modélisation des cas d'utilisation et de la conception détaillée, présentées dans le chapitre suivant.

Chapitre 3 — Conception de la solution

3.1 Introduction

Ce chapitre présente la conception de la plateforme CFSCMS. Il regroupe l'ensemble des diagrammes de conception : le diagramme de cas d'utilisation, l'architecture logicielle globale, l'architecture générale de déploiement, le modèle de données Firestore, le diagramme de classes, ainsi que les diagrammes de séquence des scénarios les plus significatifs. Le chapitre se termine par la conception de la logique métier critique et par la charte graphique retenue.

3.2 Diagramme de cas d'utilisation

La figure suivante présente le diagramme de cas d'utilisation global du système. Deux acteurs y interviennent : l'Étudiant, et l'Administrateur, aux côtés duquel peut intervenir un Acteur — une personne habilitée à le remplacer, par exemple en son absence — partageant les mêmes grandes capacités de gestion (gérer les étudiants, les formations, les demandes, les hébergements, les paiements, et envoyer des notifications). Chaque cas d'utilisation principal comporte une relation d'inclusion (« include ») vers le cas « Authentification », préalable obligatoire à toute action. Les opérations élémentaires de gestion (rechercher, ajouter, modifier, supprimer, consulter, bloquer, éliminer) sont représentées comme des extensions (« extend ») de leur cas d'utilisation englobant. Côté étudiant, les cas « Soumettre une demande d'hébergement » et « Vérifier le paiement » comportent chacun un point d'extension conditionnel, matérialisé par une note : l'envoi d'un e-mail de confirmation lorsque la distance du domicile est supérieure ou égale à trente kilomètres, et la réception de l'affectation de chambre lorsque le paiement est confirmé.

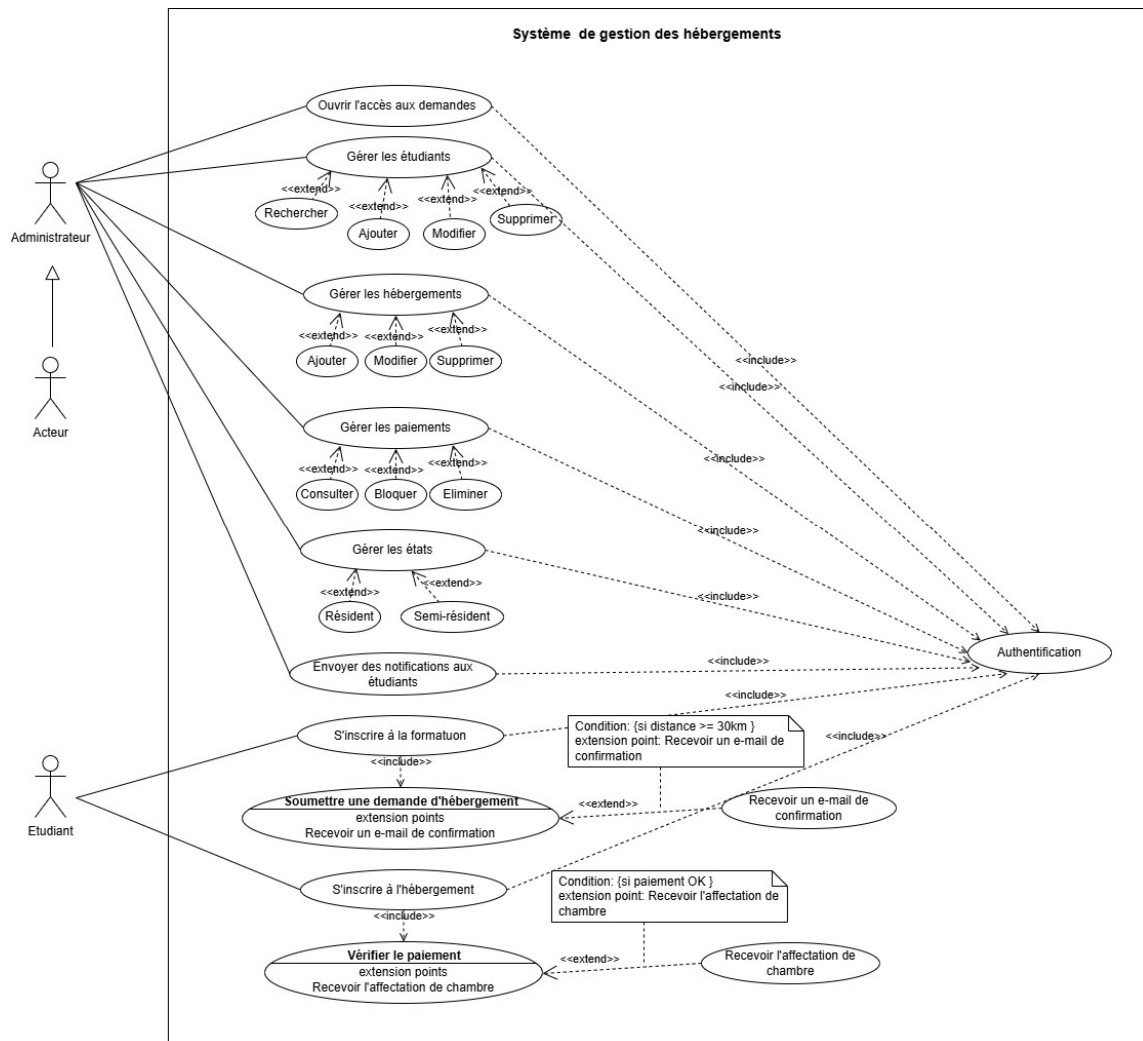


Figure 1 : Diagramme de cas d'utilisation global du système

3.3 Description des cas d'utilisation

Le tableau suivant détaille les cas d'utilisation de l'Administrateur, également accessibles à l'Acteur qui le remplace.

Tableau 4 : Cas d'utilisation — Administrateur

Cas d'utilisation	Description
Ouvrir l'accès aux demandes	Activer la période durant laquelle les étudiants peuvent soumettre une demande d'hébergement.
Gérer les étudiants	Rechercher, ajouter, modifier ou supprimer une fiche étudiant (cas étendus).
Gérer les hébergements	Ajouter, modifier ou supprimer une demande d'hébergement, avec

Cas d'utilisation	Description
	vérification de l'éligibilité et affectation de chambre (cas étendus).
Gérer les paiements	Consulter, bloquer ou éliminer un paiement (cas étendus).
Gérer les états	Attribuer l'état Résident ou Semi-résident à un étudiant (cas étendus), en cohérence avec son hébergement.
Envoyer des notifications aux étudiants	Rédiger et transmettre une notification à un ou plusieurs étudiants.

Le tableau suivant détaille les cas d'utilisation de l'acteur Étudiant.

Tableau 5 : Cas d'utilisation — Étudiant

Cas d'utilisation	Description
S'inscrire à la formation	Consulter les formations ouvertes et s'inscrire à une spécialité ; ce cas inclut « Soumettre une demande d'hébergement ».
Soumettre une demande d'hébergement	Renseigner le type d'hébergement, les dates et le motif ; point d'extension « Recevoir un e-mail de confirmation » si la distance domicile-centre est supérieure ou égale à 30 km.
S'inscrire à l'hébergement	Confirmer la demande d'hébergement ; ce cas inclut « Vérifier le paiement ».
Vérifier le paiement	Contrôler que le paiement requis est confirmé ; point d'extension « Recevoir l'affectation de chambre » si le paiement est validé.

3.4 Scénarios détaillés de cas d'utilisation

Afin d'illustrer plus finement le déroulement de certains cas d'utilisation clés, les tableaux suivants détaillent respectivement le scénario « S'inscrire à la formation » et le scénario « Gérer les hébergements (Modifier) », en précisant leurs pré-conditions, leurs post-conditions, leur scénario nominal ainsi que les scénarios alternatifs et les exceptions associés.

Tableau 6 : Scénario du cas d'utilisation « S'inscrire à la formation »

Élément	Description
Cas d'utilisation	S'inscrire à la formation

Élément	Description
Acteur	Étudiant
Pré-conditions	L'étudiant est authentifié ; la formation est ouverte et dispose de places disponibles.
Post-conditions	L'inscription est enregistrée ; un paiement d'inscription « En attente » est créé ; le cas « Soumettre une demande d'hébergement » est inclus.
Scénario nominal	1. L'étudiant ouvre l'onglet « Formations ». 2. Le système affiche la liste des formations ouvertes avec les places disponibles. 3. L'étudiant sélectionne sa spécialité et clique sur « S'inscrire ». 4. Le système inclut le cas « Soumettre une demande d'hébergement » et enregistre l'inscription ainsi que le paiement d'inscription dans Firestore. 5. Si la distance du domicile de l'étudiant est supérieure ou égale à 30 km, le point d'extension « Recevoir un e-mail de confirmation » se déclenche.
Scénario alternatif	La formation n'a plus de places disponibles ou est fermée : le système masque le bouton d'inscription et en informe l'étudiant.
Exception	Perte de connexion : le système affiche un message d'erreur et l'inscription n'est pas enregistrée.

Tableau 7 : Scénario du cas d'utilisation « Gérer les hébergements (Modifier) »

Élément	Description
Cas d'utilisation	Gérer les hébergements — cas étendu « Modifier »
Acteur	Administrateur
Pré-conditions	L'administrateur est authentifié ; une demande d'hébergement est en attente.
Post-conditions	La demande est approuvée ou rejetée ; en cas d'approbation, la chambre et l'état de l'étudiant sont mis à jour et les échéances de paiement sont générées.
Scénario nominal	1. L'administrateur ouvre la section « Hébergements », onglet « En attente ». 2. Il ouvre la demande ; le système affiche le panneau d'éligibilité calculé

Élément	Description
	automatiquement (distance routière, paiement confirmé). 3. Il clique sur « Approuver » et choisit une chambre disponible compatible avec le genre de l'étudiant. 4. Le système exécute une transaction atomique : la demande est approuvée, la chambre est mise à jour et le cas « Gérer les états » attribue à l'étudiant l'état « Résident » (ou « Semi-résident » selon la formule). 5. L'application mobile de l'étudiant est mise à jour instantanément.
Scénario alternatif	L'administrateur rejette la demande en indiquant le motif ; l'étudiant en est informé en temps réel.
Exception	Aucune chambre disponible : le système empêche l'attribution et invite l'administrateur à ajouter une chambre ou à effectuer un transfert.

3.5 Diagramme de classes

Le diagramme de classes ci-dessous modélise les onze classes du domaine et leurs relations. La classe Utilisateur porte les attributs communs à tout compte (identifiant, nom, e-mail, mot de passe, rôle) et la méthode authentifier(). Elle est spécialisée par généralisation en Etudiant, qui porte l'ensemble du profil du stagiaire, en Administrateur, et en Acteur. La classe Acteur représente ici une personne habilitée à remplacer l'administrateur — par exemple un collaborateur ou un responsable suppléant — et disposant à ce titre des mêmes capacités de gestion (gererEtudiants, gererFormations, gererDemandes, gererHebergements, gererPaiements, envoyerNotifications) ; ce partage du même contrat de méthodes entre les deux classes garantit la continuité du service en cas d'absence du titulaire du compte administrateur. Autour de ce noyau gravitent Formation, FicheEtudiant (le document imprimable de l'étudiant), Hebergement, Chambre, Etat (les états Résident et Semi-résident attribués à l'étudiant), Paiement et Notification.

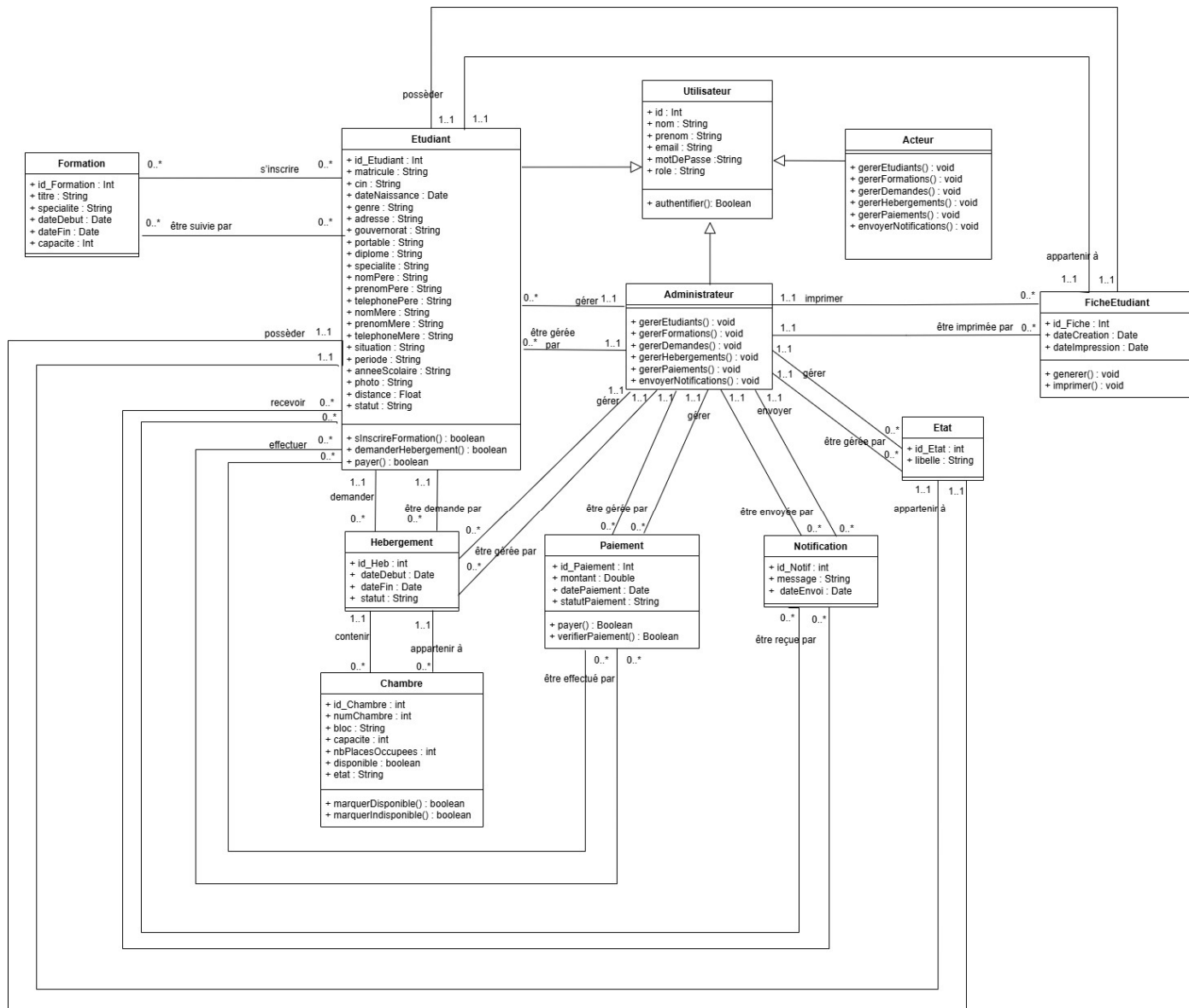


Figure 2 : Diagramme de classes de l'application CFSCMS

Le tableau suivant détaille les attributs clés et les méthodes notables de chaque classe.

Tableau 8 : Attributs et méthodes notables des classes du modèle

Classe	Attributs clés	Méthodes notables
Utilisateur	id, nom, prenom, email, motDePasse, role	authentifier() : Boolean
Etudiant (hérite de Utilisateur)	matricule, cin, dateNaissance, genre, adresse, gouvernorat, distance, statut...	sInscrireFormation(), demanderHebergement(), payer()
Acteur	—	gererEtudiants(),

Classe	Attributs clés	Méthodes notables
		gererFormations(), gererDemandes(), gererHebergements(), gererPaielements(), envoyerNotifications()
Administrateur (hérite de Utilisateur)	—	gererEtudiants(), gererFormations(), gererDemandes(), gererHebergements(), gererPaielements(), envoyerNotifications()
Formation	titre, specialite, dateDebut, dateFin, capacite	—
FicheEtudiant	dateCreation, dateImpression	generer(), imprimer()
Hebergement	dateDebut, dateFin, statut	—
Chambre	numChambre, bloc, capacite, nbPlacesOccupees, disponible, etat	marquerDisponible(), marquerIndisponible()
Etat	libelle	—
Paielement	montant, datePaielement, statutPaielement	payer(), verifierPaielement()
Notification	message, dateEnvoi	—

3.6 Vues dynamiques : les diagrammes de séquence

Un diagramme de séquence montre les interactions chronologiques entre les acteurs et les composants du système lors de l'exécution d'un scénario. Pour mieux comprendre le fonctionnement de la plateforme, nous présentons les diagrammes de séquence des trois scénarios les plus significatifs.

3.6.1 Diagramme de séquence « S'authentifier »

La figure suivante illustre le processus d'authentification : l'utilisateur saisit son adresse e-mail et son mot de passe, les identifiants sont vérifiés par Firebase Authentication, puis le

rôle est lu dans la collection « utilisateurs » afin de charger l'espace correspondant — interface web d'administration ou tableau de bord mobile de l'étudiant.

Diagramme de séquence — « S'authentifier »

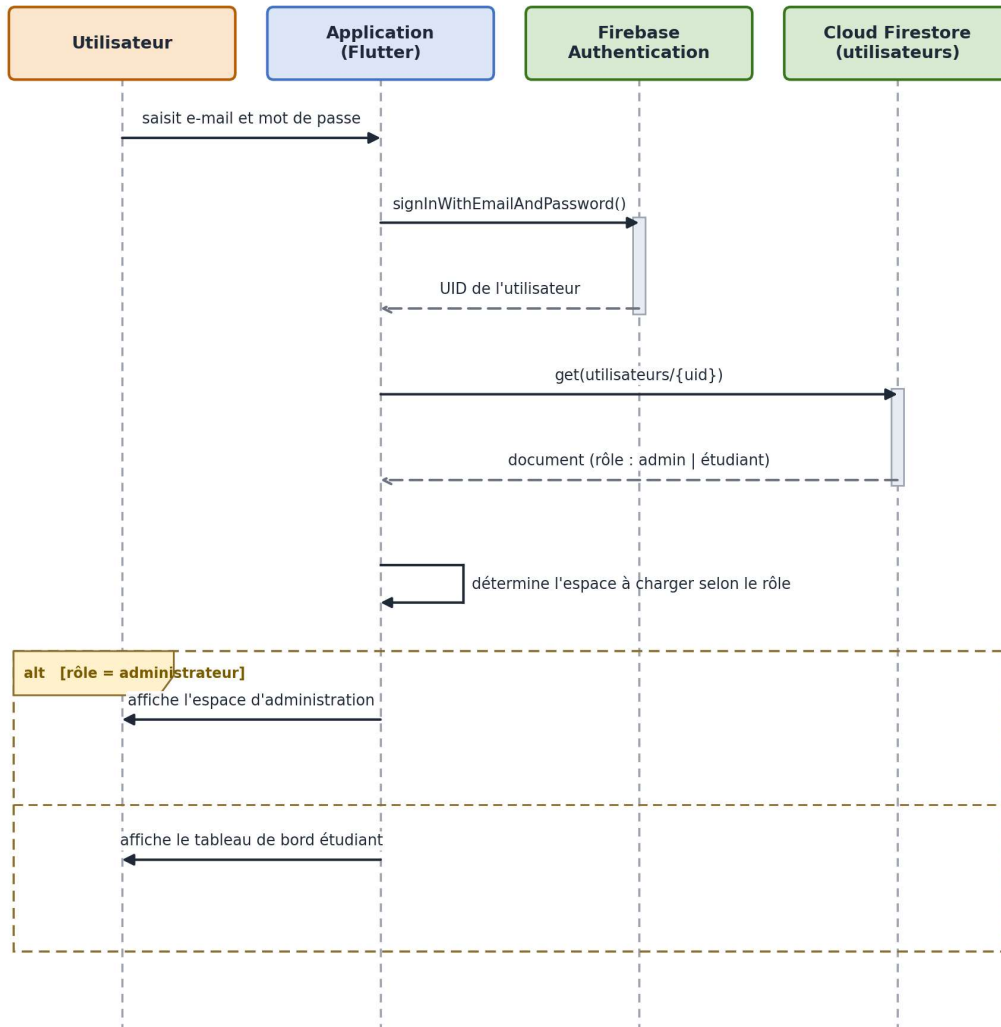


Figure 3 : Diagramme de séquence « S'authentifier »

3.6.2 Diagramme de séquence « Soumettre une demande d'hébergement »

La figure suivante illustre la soumission d'une demande : l'application vérifie automatiquement les critères d'éligibilité de l'étudiant (inscription active, paiement confirmé, distance supérieure à trente kilomètres) ; si ces critères sont satisfaits, le formulaire devient accessible et la demande est enregistrée dans Firestore avec le statut « En attente », visible instantanément côté administration ; dans le cas contraire, le critère manquant est signalé à l'étudiant.

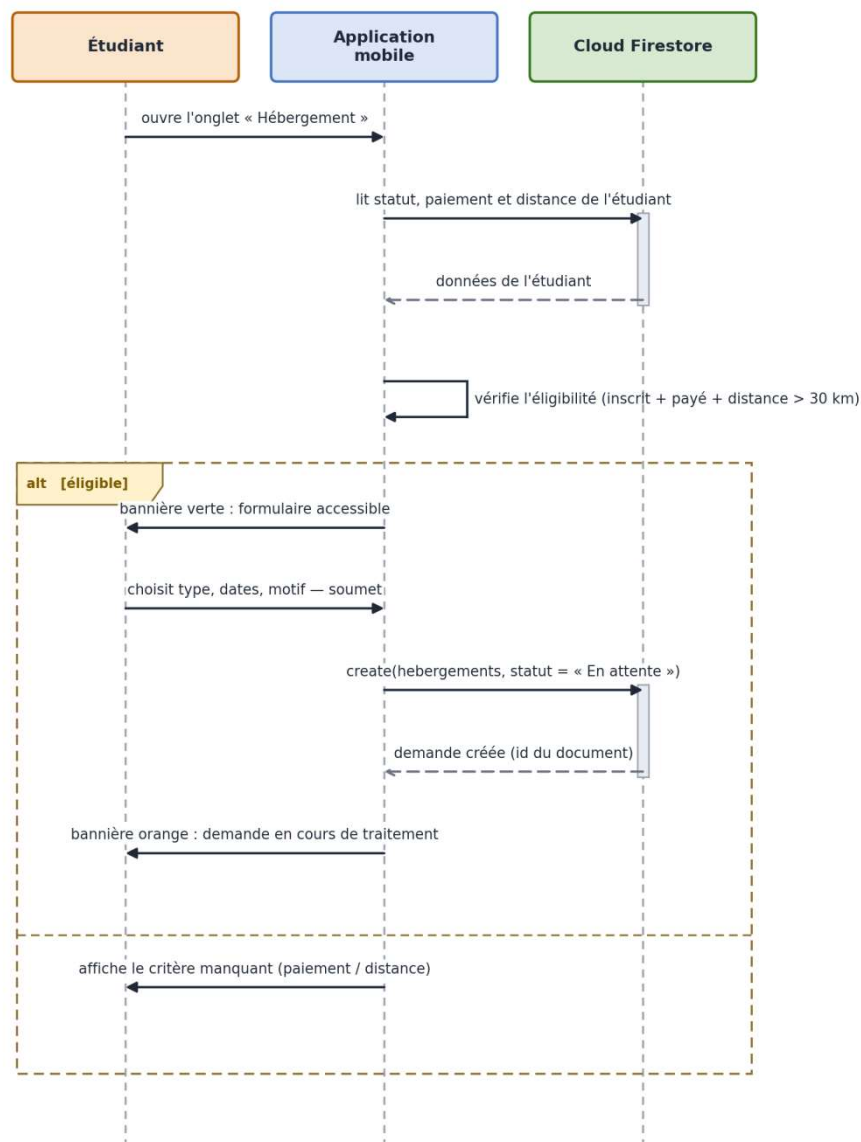
Diagramme de séquence — « Soumettre une demande d'hébergement »

Figure 4 : Diagramme de séquence « Soumettre une demande d'hébergement »

3.6.3 Diagramme de séquence « Traiter une demande d'hébergement »

La figure suivante illustre le traitement d'une demande côté administration : consultation du panneau d'éligibilité, approbation, choix de la chambre, puis exécution d'une transaction atomique Firestore mettant à jour simultanément la demande, la chambre et le statut de l'étudiant ; la propagation temps réel des changements vers l'application mobile de l'étudiant concerné, via les flux Firestore, est également représentée.

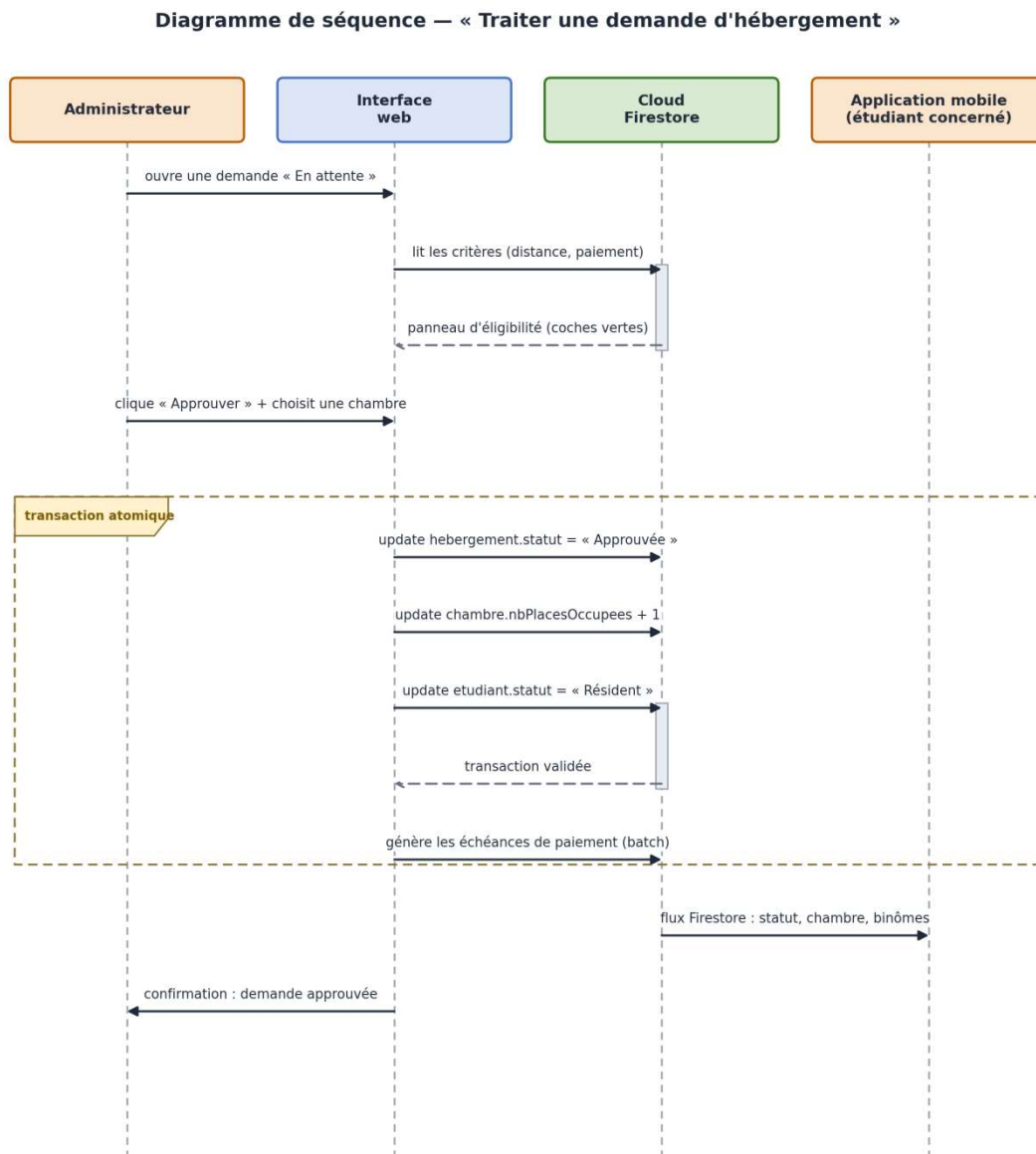


Figure 5 : Diagramme de séquence « Traiter une demande d'hébergement »

3.7 Architecture logicielle globale

L'application est structurée en couches selon une séparation claire des responsabilités, ce qui facilite la maintenance et l'évolution du code. La figure suivante illustre cette architecture, depuis l'interface utilisateur jusqu'à l'infrastructure Firebase.

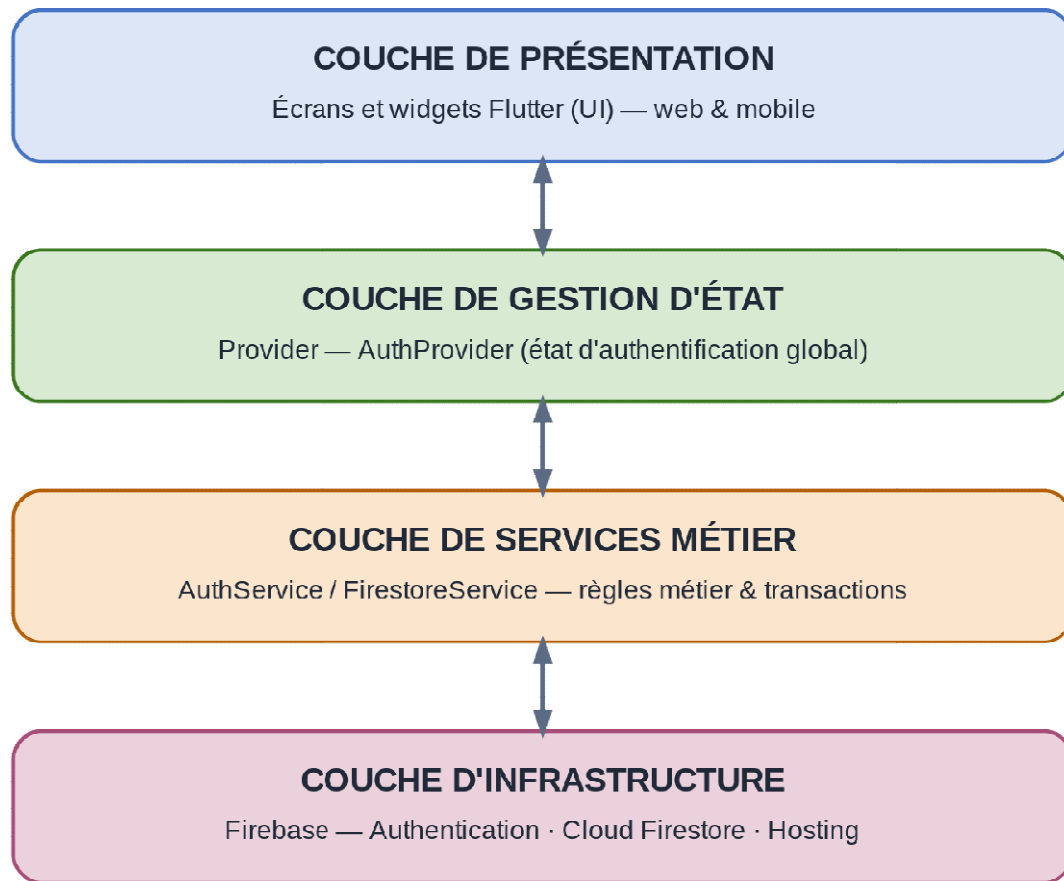


Figure 6 : Architecture logicielle en couches de l'application CFSCMS

La couche de présentation regroupe les écrans (screens) et les widgets réutilisables développés en Flutter, communs autant que possible aux deux cibles (web et mobile). La couche de gestion d'état, implémentée avec le package Provider, expose l'état d'authentification (AuthProvider) à l'ensemble de l'arbre de widgets et déclenche la reconstruction de l'interface lors des changements d'état. La couche de services encapsule toute la logique d'accès aux données : AuthService pour l'authentification, FirestoreService pour l'ensemble des opérations sur la base de données, y compris les transactions et les écritures groupées. Enfin, la couche d'infrastructure repose sur les services Firebase : Firebase Authentication pour la gestion des comptes, Cloud Firestore comme base de données NoSQL temps réel, et Firebase Hosting pour la publication de la version web.

3.8 Architecture générale et environnement de déploiement

La plateforme est une solution multiplateforme : l'interface d'administration est compilée en version web et déployée sur Firebase Hosting, accessible depuis n'importe quel navigateur, tandis que l'application étudiante est compilée sous forme d'APK signé, distribué aux

étudiants et installé sur leurs smartphones Android. La figure suivante illustre cette architecture de déploiement.

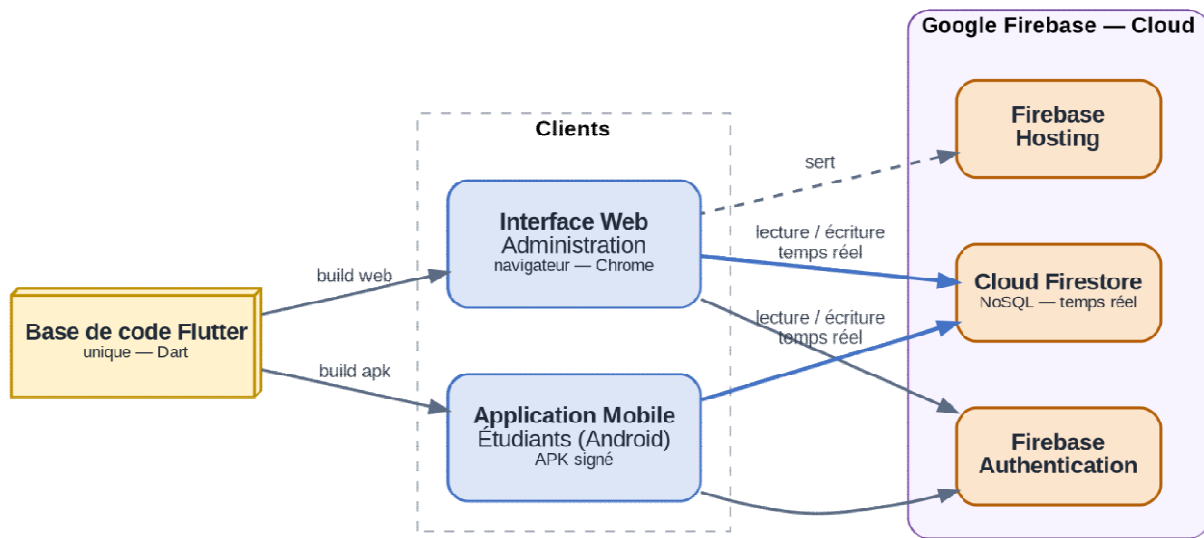


Figure 7 : Architecture générale et environnement de déploiement

Les deux clients communiquent avec les mêmes services Firebase : Firebase Authentication pour l'authentification des utilisateurs, Cloud Firestore pour la base de données temps réel, et Firebase Hosting pour la diffusion de la version web. Les règles de sécurité Firestore, publiées depuis la console, garantissent le cloisonnement des données : l'étudiant n'accède qu'à ses propres documents, tandis que les opérations de gestion sont réservées au rôle administrateur.

3.9 Modèle de données : les collections Firestore

La base de données Cloud Firestore est une base NoSQL orientée documents, organisée en collections. Contrairement au modèle relationnel, les données n'y sont pas réparties en tables reliées par des clés étrangères strictes, mais en collections de documents dont la cohérence est assurée applicativement, notamment par des transactions. La classe Etudiant, tout comme la classe Administrateur, spécialise par généralisation la classe Utilisateur (qui porte l'identifiant, le nom, l'e-mail, le mot de passe et le rôle communs à tout compte) ; en base, cette généralisation se traduit par une collection utilisateurs commune, complétée par une collection étudiants qui porte les attributs propres au profil étudiant. Le tableau suivant présente les principales collections retenues, leurs attributs clés et leurs relations avec les autres collections.

Tableau 9 : Description des collections Firestore

Collection	Attributs principaux	Relations
utilisateurs	id, nom, prenom, email, motDePasse, role	Généralisée par étudiants ; rôle « administrateur » géré sans collection dédiée
etudiants	matricule, cin, dateNaissance, genre, adresse, gouvernorat, portable, diplome, specialite, filiation (nomPère/nomMère...), situation, periode, anneeScolaire, photo, distance, statut	Hérite d'utilisateurs ; liée à formations, hébergements, paiements, notifications, fichesEtudiants
formations	titre, specialite, dateDebut, dateFin, capacite	Référencée par étudiants (s'inscrire)
fichesEtudiants	dateCreation, dateImpression	Possédée par un étudiant (relation 1..1)
hebergements	dateDebut, dateFin, statut	Demandée par un étudiant, rattachée à une chambre
chambres	numChambre, bloc, capacite, nbPlacesOccupees, disponible, etat	Contient des hébergements ; liée à etats
etats	libelle (Résident, Semi-résident)	Attribuée par l'administrateur, liée à chambres
paiements	montant, datePaiement, statutPaiement	Effectué par un étudiant
notifications	message, dateEnvoi	Reçue par un étudiant

La sécurité de ce modèle est garantie par les règles Firestore : chaque étudiant ne peut lire que les documents qui lui appartiennent (sa fiche, ses demandes, ses paiements, ses notifications), il ne peut créer qu'une demande d'hébergement à son propre nom avec le statut initial « En attente », tandis que toutes les opérations de gestion sont réservées au rôle administrateur.

3.10 Conception de la logique métier critique

3.10.1 Éligibilité à l'hébergement

Un étudiant est considéré éligible à une demande d'hébergement lorsque deux conditions sont réunies simultanément : son statut actuel est « Non inscrit », « Inscrit » ou « Externe », c'est-à-dire qu'il ne dispose pas déjà d'un hébergement actif, et il est rattaché à une formation existante dans le système, avec une distance domicile-centre supérieure au seuil de trente kilomètres. Cette règle est encapsulée dans une propriété calculée du modèle étudiant, garantissant qu'elle est évaluée de manière identique partout où elle est utilisée dans l'application, aussi bien côté administrateur (panneau d'éligibilité) que côté étudiant (accès au formulaire de demande).

3.10.2 Inscription à une formation

Lorsqu'un étudiant s'inscrit à une formation, le système exécute une transaction Firestore qui vérifie d'abord que la capacité de la formation n'est pas atteinte, incrémente le compteur d'inscrits, met à jour le statut de l'étudiant, puis, si des frais d'inscription sont configurés pour la formation, crée automatiquement un enregistrement de paiement au statut « En attente ». Conformément au diagramme de cas d'utilisation, ce cas « S'inscrire à la formation » inclut le cas « Soumettre une demande d'hébergement » : le système évalue dans la foulée l'éligibilité de l'étudiant à l'hébergement et, si sa distance au centre est supérieure ou égale à trente kilomètres, déclenche l'envoi d'un e-mail de confirmation. L'utilisation d'une transaction garantit qu'aucune double inscription ne peut dépasser la capacité, même en cas de requêtes concurrentes émises par plusieurs étudiants au même instant.

3.10.3 Approbation d'une demande d'hébergement

L'approbation d'une demande d'hébergement est l'opération la plus sensible du système, car elle doit maintenir la cohérence de trois entités liées : la demande d'hébergement elle-même, la chambre attribuée et l'état de l'étudiant. Le traitement se déroule en deux temps. Une transaction atomique met d'abord à jour simultanément le statut de la demande (« Approuvée »), incrémente le nombre de places occupées de la chambre concernée et recalcule sa disponibilité, puis attribue à l'étudiant l'état Résident ou Semi-résident (classe Etat) selon le type d'hébergement choisi — ce qui correspond au cas d'utilisation « Gérer les états ». Dans un second temps, le système génère automatiquement, mois par mois, les échéances de

paiement correspondant à la durée de l'hébergement, en évitant toute génération en double si l'opération venait à être rejouée.

3.10.4 Gestion de l'occupation des chambres

Le transfert d'un étudiant d'une chambre à une autre, ou l'échange entre deux étudiants, s'appuie également sur des transactions ou des écritures groupées (batch), afin que les compteurs d'occupation des chambres source et destination restent toujours synchronisés avec la réalité, y compris en cas d'opérations simultanées effectuées par plusieurs utilisateurs de l'interface d'administration. Aucun compteur n'est jamais fixé a priori : il est systématiquement recalculé à travers l'opération transactionnelle portant à la fois sur le document source et sur le document lié.

3.11 Charte graphique et composants d'interface

L'interface visuelle repose sur une police de caractères moderne (Poppins, chargée via Google Fonts) et une palette de couleurs centralisée, cohérente entre l'interface web et l'application mobile. Un ensemble de composants réutilisables, parmi lesquels des cartes premium, des badges de statut colorés, des indicateurs chiffrés et une superposition de chargement, assure une cohérence visuelle entre l'ensemble des écrans, aussi bien côté administrateur que côté étudiant.

3.12 Conclusion

Ce chapitre a présenté la conception détaillée de la plateforme CFSCMS : la modélisation des cas d'utilisation, l'architecture logicielle en couches, l'architecture de déploiement, le modèle de données Firestore, le diagramme de classes, les diagrammes de séquence des scénarios critiques ainsi que la logique métier sensible qui garantit la cohérence des données. Cette conception constitue la base sur laquelle s'appuie la phase de réalisation, présentée après un chapitre consacré à l'environnement de travail et aux choix technologiques retenus.

Chapitre 4 — Environnement de travail et choix technologiques

4.1 Introduction

Ce chapitre décrit l'environnement matériel et logiciel utilisé pour la réalisation de la plateforme, justifie les principaux choix technologiques retenus, et dresse la liste des bibliothèques et dépendances qui composent le projet.

4.2 Environnement matériel

L'implémentation de la plateforme a été réalisée sur un ordinateur portable dont les caractéristiques sont décrites au tableau 10.

Tableau 10 : Environnement matériel

Élément	Caractéristique
Marque	Asus F556U
Processeur	Intel(R) Core(TM) i7-7500U CPU @ 2.70 GHz — 2.90 GHz
Disque dur	223 Go
RAM	12,0 Go
Système d'exploitation	Windows 10 Famille
Terminal de test mobile	Smartphone Android (installation de l'APK de test)

4.3 Environnement logiciel

Le tableau 11 présente les logiciels utilisés pour la réalisation de la plateforme.

Tableau 11 : Logiciels utilisés

Logiciel	Description
Visual Studio Code	Éditeur de code source multiplateforme, enrichi des extensions Flutter et Dart pour le développement, le débogage et le rechargement à chaud (hot reload).

Logiciel	Description
Android Studio / SDK Android	Fournit le SDK Android, les émulateurs, ainsi que les outils de génération et de signature des APK.
Console Firebase	Interface web de Google permettant de configurer l'authentification, la base Firestore, les règles de sécurité et l'hébergement web.
Firebase CLI / FlutterFire CLI	Outils en ligne de commande utilisés pour lier le projet Flutter au projet Firebase et pour déployer la version web sur Firebase Hosting.
Draw.io	Outil de diagrammation utilisé pour la conception des diagrammes UML (cas d'utilisation, classes, séquence, déploiement).

4.4 Choix technologiques et justification

4.4.1 Flutter

Flutter a été retenu pour permettre le développement d'une application unique, exécutable à la fois en tant que site web pour l'administration et en tant qu'application Android pour les étudiants, à partir d'une seule base de code Dart. Ce choix limite la duplication d'effort et garantit une expérience visuelle cohérente entre les deux plateformes, tout en bénéficiant d'un écosystème mature et d'une large communauté.

4.4.2 Firebase comme backend

L'utilisation de Firebase, en tant que Backend-as-a-Service (BaaS), permet de s'affranchir de la mise en place et de la maintenance d'un serveur applicatif dédié. Cloud Firestore, en particulier, offre nativement la synchronisation temps réel des données via des flux (streams), ce qui répond directement au besoin non fonctionnel de mise à jour instantanée entre l'interface administrateur et l'interface étudiant. Firebase Authentication prend en charge la gestion sécurisée des comptes et des sessions, tandis que Firebase Hosting permet un déploiement simple et rapide de la version web.

4.4.3 Provider pour la gestion d'état

Le package Provider a été choisi pour la gestion de l'état global de l'application, notamment l'état d'authentification. Il s'agit d'une solution légère, largement adoptée dans

l'écosystème Flutter, suffisante au regard de la complexité de l'état à gérer dans ce projet, par comparaison avec des solutions plus lourdes telles que Bloc ou Riverpod.

4.5 Bibliothèques et dépendances principales

Le tableau 12 dresse la liste des principaux packages Dart / Flutter utilisés dans le projet.

Tableau 12 : Bibliothèques et dépendances principales

Package	Version	Rôle
firebase_core	^4.10.0	Initialisation de Firebase dans l'application.
firebase_auth	^6.5.2	Authentification (connexion, réinitialisation de mot de passe).
cloud_firestore	^6.5.0	Base de données NoSQL temps réel.
provider	^6.1.5+1	Gestion d'état global.
google_fonts	^8.1.0	Typographie de l'identité visuelle (Poppins).
pdf	^3.11.0	Génération de documents PDF.
printing	^5.13.1	Impression et partage de documents PDF.
image_picker	^1.2.2	Sélection de la photo de profil étudiant.
url_launcher	^6.3.1	Envoi d'e-mails depuis l'application.
intl	^0.20.2	Formatage des dates et des nombres.

4.6 Conclusion

Ce chapitre a présenté l'environnement matériel et logiciel mobilisé pour la réalisation du projet, ainsi que la justification des principaux choix technologiques : Flutter pour le développement multiplateforme, Firebase comme backend temps réel, et Provider pour la gestion d'état. Le chapitre suivant décrit la réalisation concrète des différents modules de la plateforme.

Chapitre 5 — Réalisation

5.1 Introduction

La phase de réalisation vient concrétiser les efforts déployés au cours des chapitres précédents. Ce chapitre décrit d'abord l'organisation du code source et le mécanisme d'authentification, puis illustre les fonctionnalités développées à travers les interfaces de la plateforme, en suivant le scénario réel d'utilisation : le parcours de l'administrateur, puis celui de l'étudiant. Il se termine par la présentation du mécanisme de génération de documents PDF et des dispositifs de sécurité des données.

5.2 Organisation du code source

Le projet est organisé selon une arborescence par responsabilité, séparant clairement les constantes et le thème (core), les modèles de données (models), la gestion d'état (providers), l'accès aux données (services), les écrans (screens, eux-mêmes répartis en sous-dossiers auth, admin et etudiant) et enfin les composants réutilisables (widgets). Cette organisation facilite la navigation dans le code, la relecture par un tiers ainsi que l'ajout de nouveaux modules sans remettre en cause l'existant.

5.3 Authentification et gestion des rôles

Au lancement de l'application, AuthProvider détermine si un utilisateur est déjà connecté. Après connexion, le rôle stocké dans la collection utilisateurs (administrateur ou étudiant) détermine la redirection automatique vers l'interface correspondante : l'espace d'administration (admin_layout) ou l'espace stagiaire (etudiant_shell). Un lien de création de compte, désactivable avant la mise en production, permet la création initiale du compte administrateur ; en exploitation normale, seuls les comptes créés par l'administration peuvent se connecter, ce qui garantit la maîtrise des accès.

5.4 Espace administrateur (interface web)

L'administrateur ouvre l'interface web du CFSCMS et saisit ses identifiants. Après vérification par Firebase Authentication, il accède à l'espace de gestion.

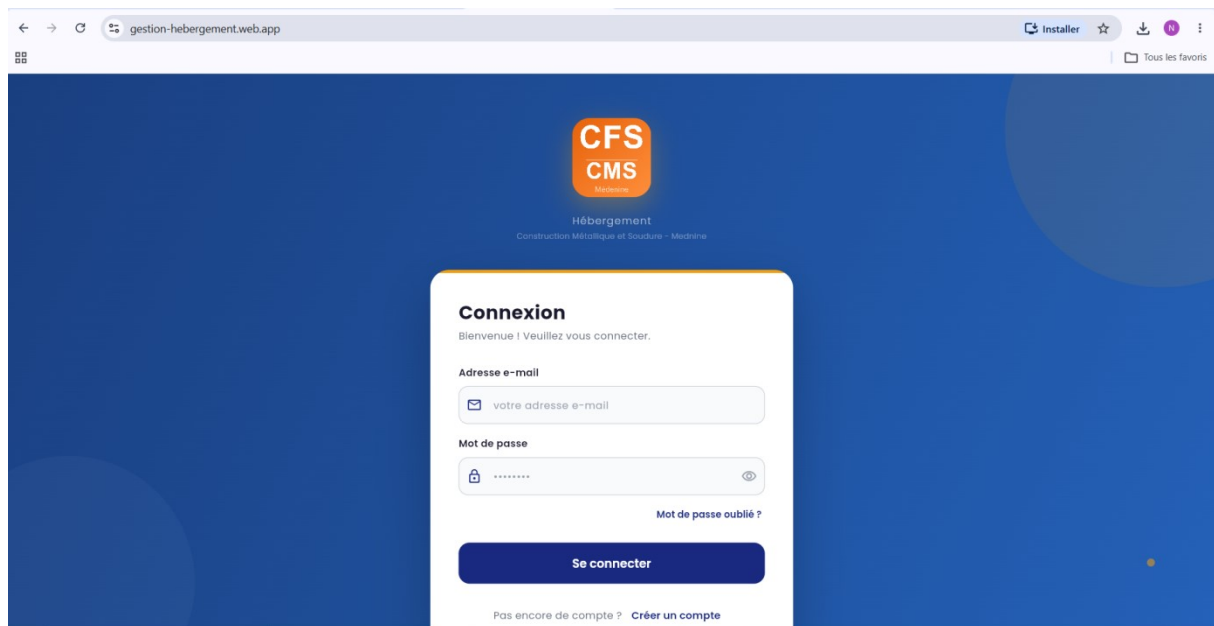


Figure 8 : Interface d'authentification de l'administrateur

Dès la connexion, le tableau de bord affiche les indicateurs clés du centre. Quatre cartes statistiques présentent le nombre total d'étudiants, le nombre de demandes d'hébergement en attente, le nombre de paiements à encaisser et le nombre de chambres disponibles, tandis qu'un graphique montre la répartition des étudiants par statut (Non inscrit, Résident, Semi-résident, Externe). Toutes les données se synchronisent automatiquement via Firestore, sans aucun rechargement de page.

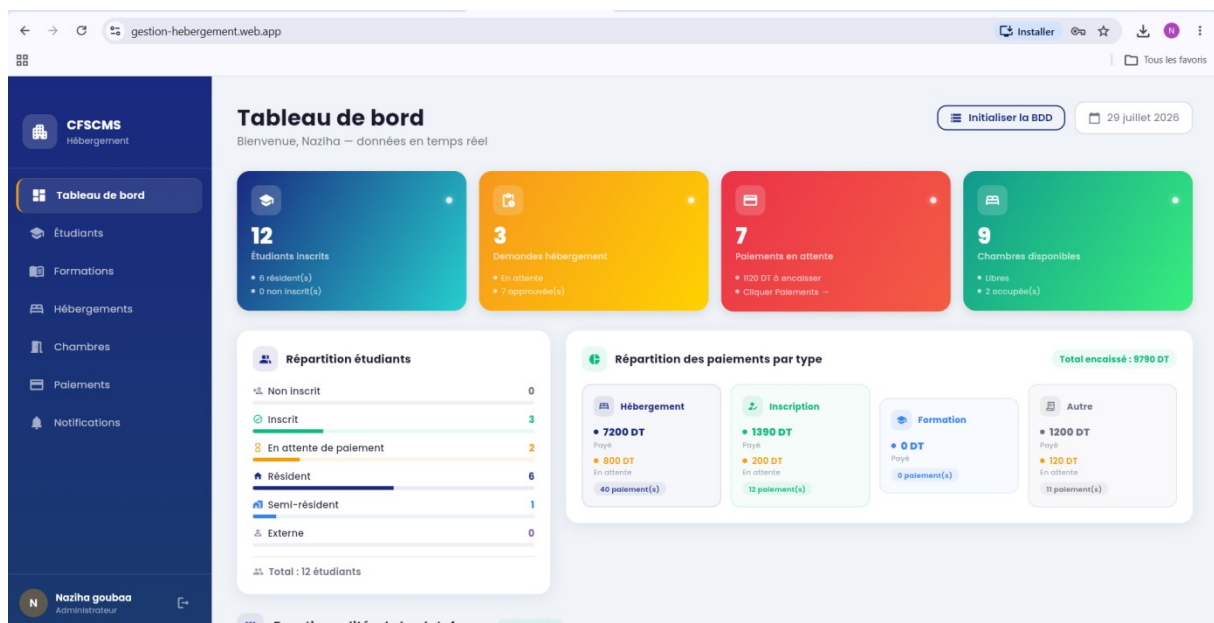


Figure 9 : Tableau de bord administrateur

La section « Étudiants » permet de consulter la liste complète des étudiants, de filtrer par statut ou de rechercher par nom. Pour ajouter un nouveau compte, l'administrateur ouvre un formulaire de création : il saisit le nom, le prénom, l'adresse e-mail, le téléphone et le gouvernorat d'origine ; le statut est automatiquement défini sur « Non inscrit » et le matricule est généré automatiquement selon les règles du centre (abréviation de la spécialité et date). Le compte est créé simultanément dans Firebase Authentication et dans Firestore, et l'étudiant apparaît immédiatement dans la liste ; l'administrateur peut ensuite transmettre les identifiants par e-mail ou sous forme de PDF. La suppression d'un étudiant entraîne, par cohérence, la suppression en cascade de ses demandes d'hébergement, de ses paiements et de ses notifications associées.

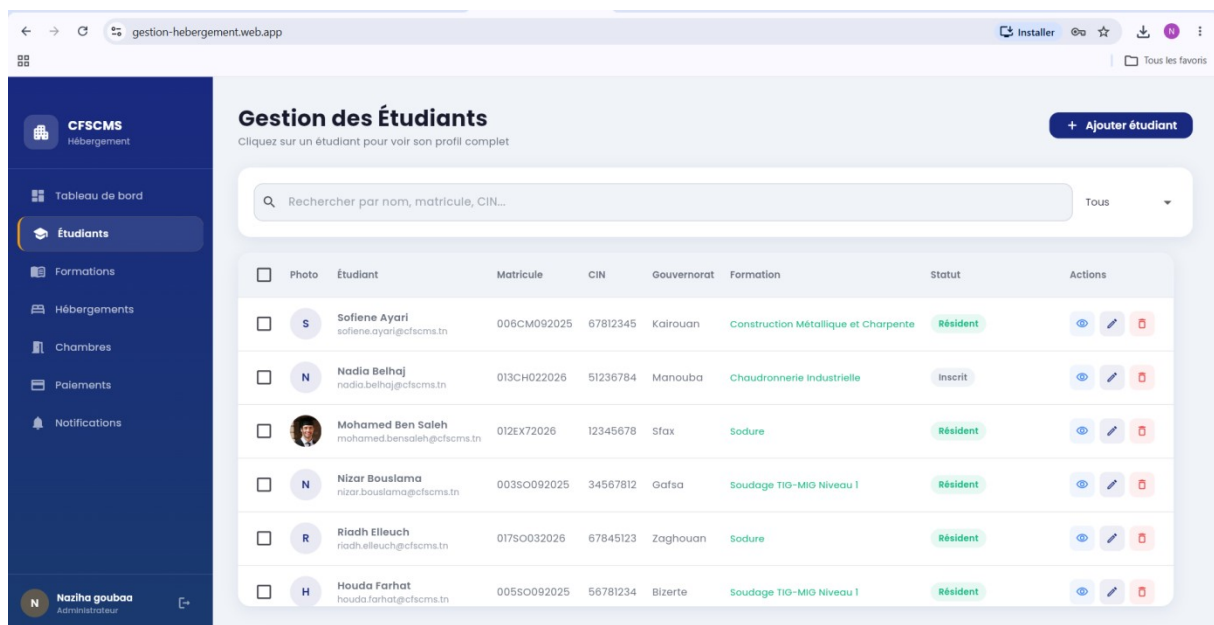


Figure 10 : Interface de gestion des étudiants

Figure 11 : Formulaire de création d'un compte étudiant

La section « Formations » permet à l'administration de créer des formations, d'en définir les tarifs (inscription, hébergement en pension complète, hébergement en demi-pension) et la capacité, et d'ouvrir l'accès aux demandes d'hébergement pour la période choisie — le cas d'utilisation « Ouvrir l'accès aux demandes » présenté au chapitre 3. Le nombre de places restantes et le taux de remplissage de chaque formation sont suivis en temps réel.

Formation	Dates	Statut	Inscription	Résident/mois	Sémi-rés/mois
Construction Métallique	13/07/26 – 11/10/26	Ouverte	150 DT	200 DT	120 DT
Soudure	01/03/26 – 31/12/26	Ouverte	120 DT	200 DT	120 DT
Découpe et Assemblage Métallique	01/02/26 – 31/12/26	Ouverte	100 DT	180 DT	110 DT
Chaudronnerie Industrielle	01/02/26 – 31/12/26	Ouverte	-	-	-
Construction Métallique et Charpente	16/09/25 – 30/06/26	Fermée	-	-	-
Soudage TIG-MIG Niveau 1	16/09/25 – 30/06/26	Fermée	-	-	-

Figure 12 : Interface de gestion des formations

Dans la section « Hébergements », l'onglet « En attente » affiche instantanément toute nouvelle demande, classée par date de demande décroissante. En ouvrant une demande, l'administrateur consulte le panneau d'éligibilité calculé automatiquement par l'application : la distance entre le gouvernorat de l'étudiant et le centre est déterminée grâce à une table de distances routières intégrée, et des coches vertes confirment que tous les critères sont satisfaits (distance suffisante et paiement confirmé).

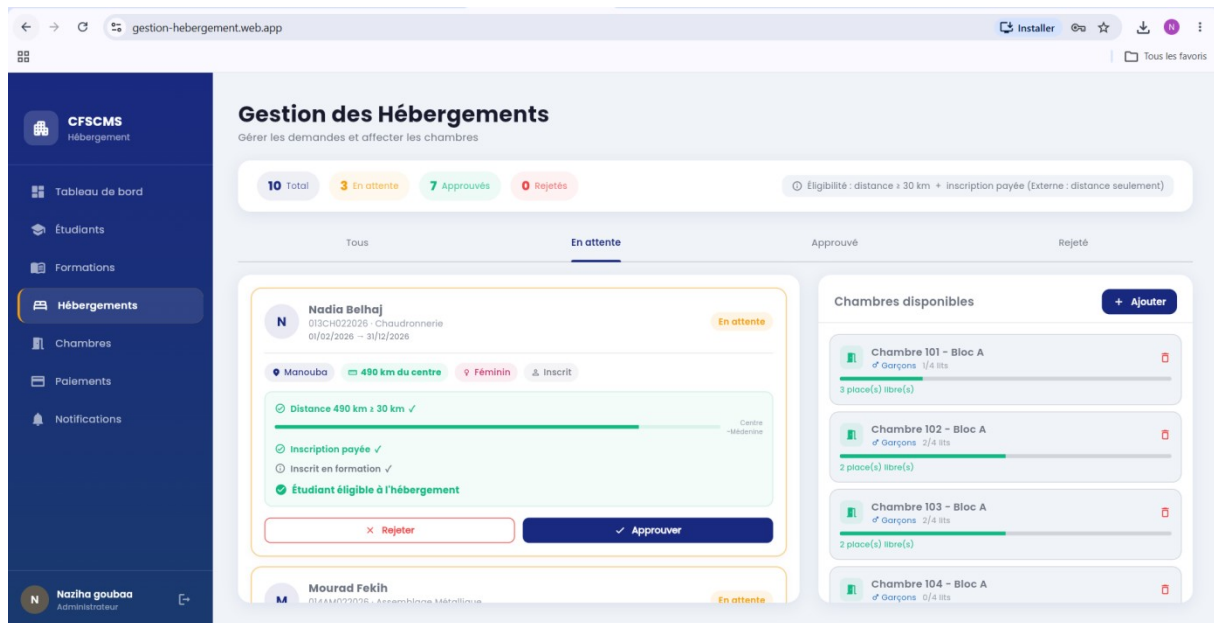


Figure 13 : Interface des demandes d'hébergement et panneau d'éligibilité

L'administrateur clique ensuite sur « Approuver » et choisit une chambre disponible dans la liste proposée, selon le genre de l'étudiant. Trois opérations s'exécutent alors simultanément dans Firestore via une transaction atomique : la demande est approuvée, la chambre est mise à jour et le statut de l'étudiant est modifié — il devient par exemple « Résident » lorsque le type choisi est la pension complète. L'administrateur peut également transférer un étudiant vers une autre chambre ou rejeter une demande en indiquant un motif.

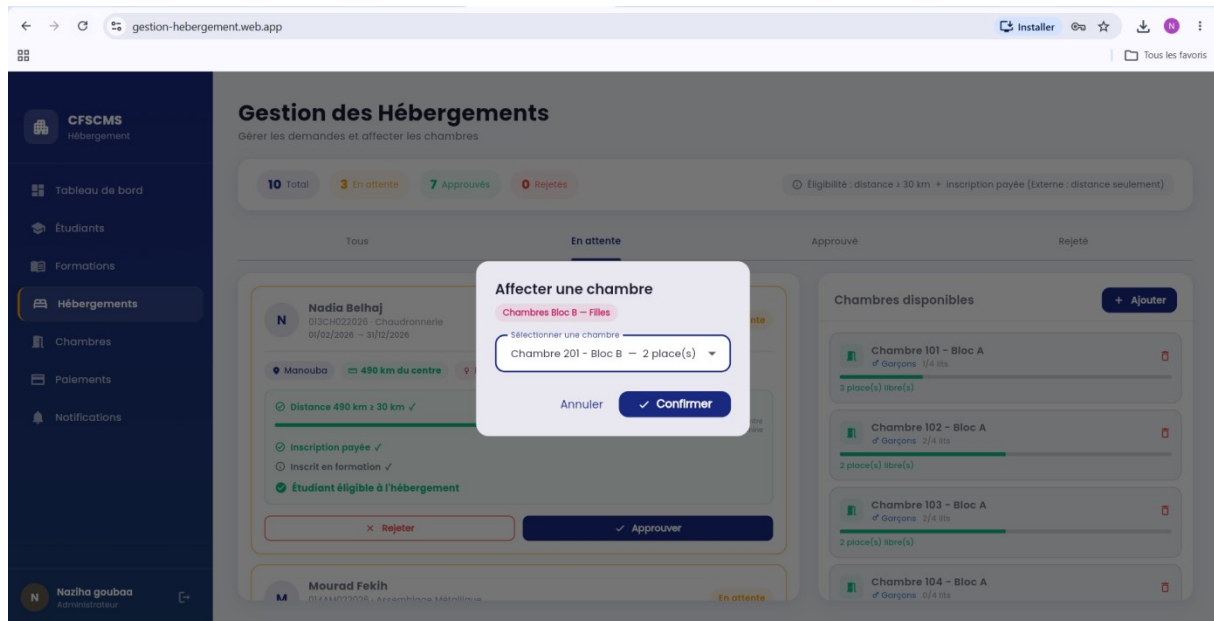


Figure 14 : Approbation d'une demande et attribution d'une chambre

La section « Chambres » présente les blocs A et B avec l'occupation en temps réel et la liste des occupants de chaque chambre. L'administrateur peut ajouter une chambre lorsqu'une nouvelle chambre est prête, et dispose d'outils de transfert d'un étudiant vers une autre chambre ou d'échange entre deux étudiants occupant des chambres différentes.

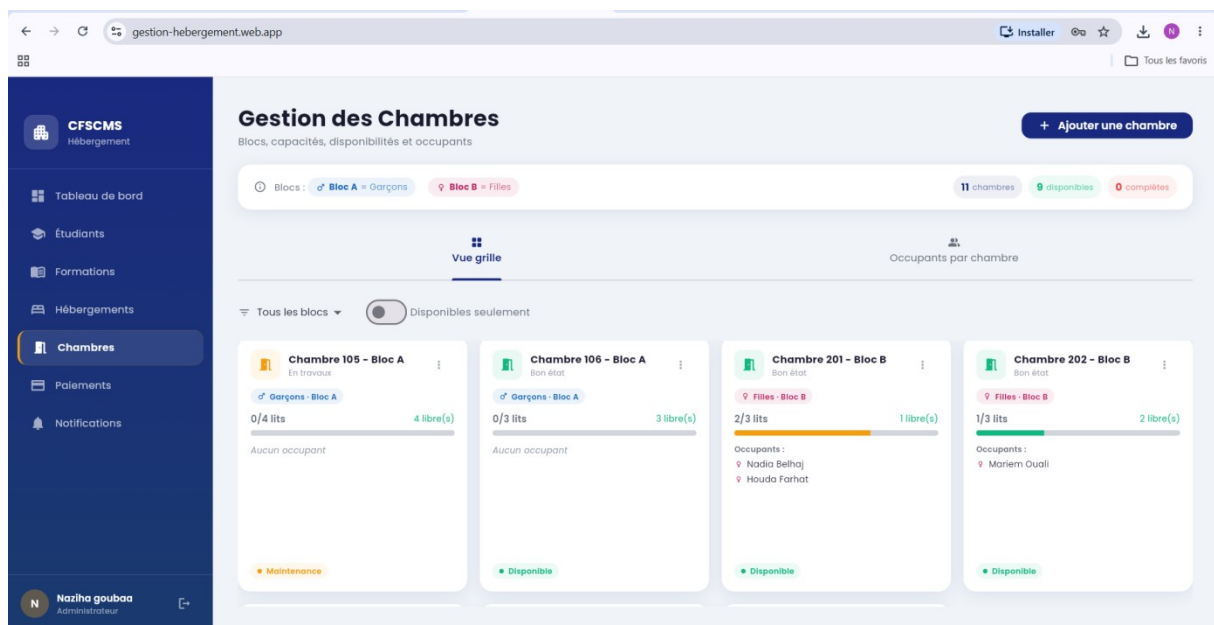


Figure 15 : Interface de gestion des chambres (blocs A et B)

La section « Paiements » affiche des cartes statistiques (montant total encaissé, paiements en attente, paiements validés du mois) et permet de filtrer les transactions. En filtrant sur « En attente », l'administrateur localise rapidement le paiement d'un étudiant et le valide d'un simple clic : le statut passe à « Payé » et le reçu officiel peut être imprimé ou téléchargé au format PDF, avec l'en-tête du centre et l'ensemble des détails de la transaction. La validation du paiement d'inscription fait passer le statut de l'étudiant à « Inscrit » et débloque son éligibilité à l'hébergement. L'administrateur peut également créer manuellement un paiement d'inscription, d'hébergement ou de mensualité via le bouton « Ajouter un paiement ».

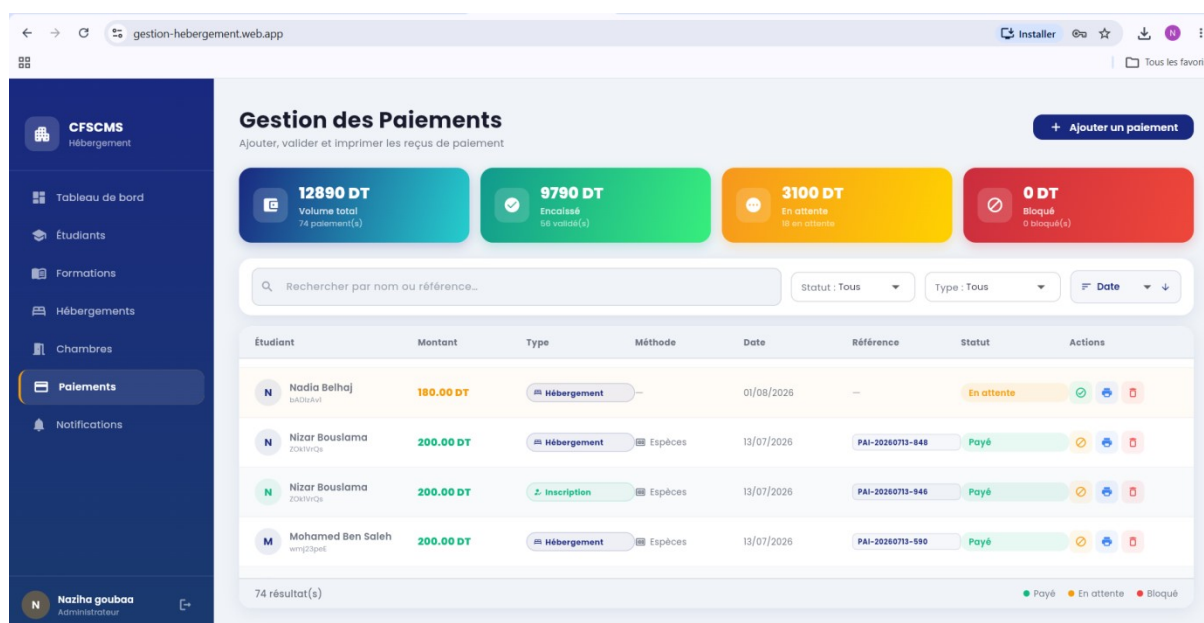


Figure 16 : Interface de gestion des paiements et reçu PDF

La section « Notifications » permet de rédiger un message destiné à un étudiant, par exemple une notification de bienvenue après la création de son compte, puis de l'envoyer via la plateforme, avec possibilité de partage par e-mail. La notification est reçue en temps réel sur l'application mobile de l'étudiant concerné.

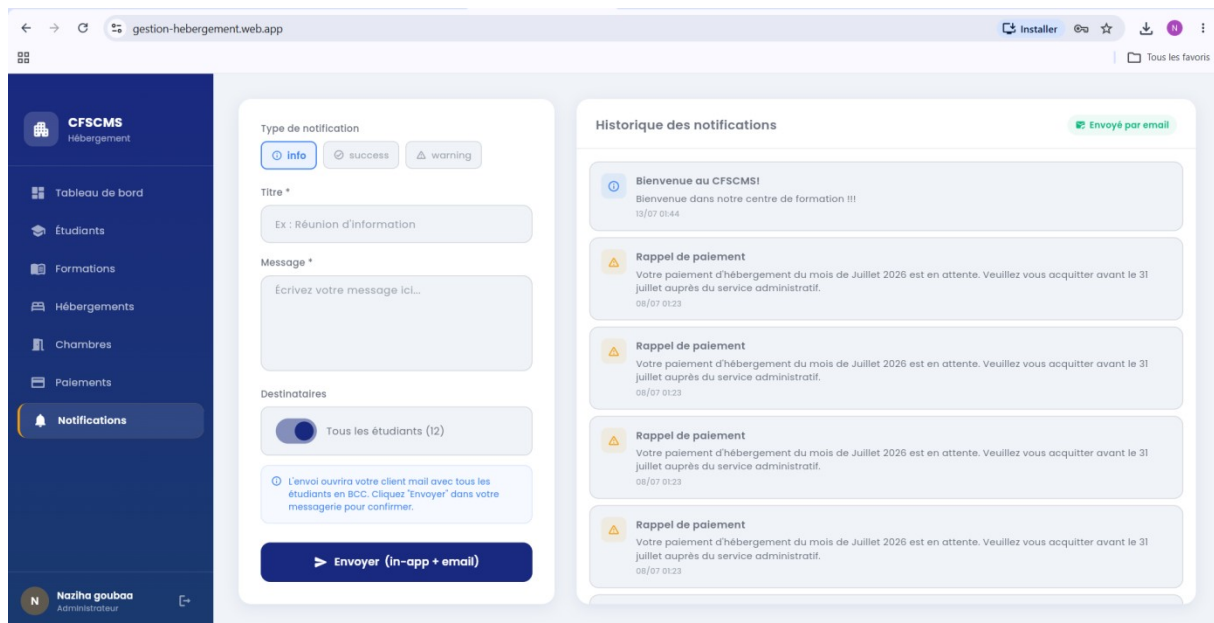


Figure 17 : Interface d'envoi des notifications

5.5 Espace étudiant (application mobile)

L'étudiant ouvre l'application mobile CFSCMS sur son smartphone, saisit l'adresse e-mail et le mot de passe transmis par l'administration, puis confirme sa connexion.

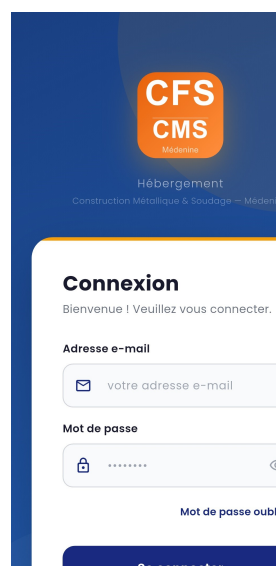


Figure 18 : Interface de connexion de l'application mobile étudiant

L'étudiant arrive ensuite sur son tableau de bord personnel, où son statut actuel est affiché clairement (initialement « Non inscrit »). Les cartes de résumé indiquent sa situation vis-à-vis de la formation et de l'hébergement, et l'icône de cloche signale les notifications non lues —

comme le message de bienvenue envoyé par l'administrateur, transmis et reçu en temps réel via Firestore.



Figure 19 : Tableau de bord personnel de l'étudiant

Dès la première connexion, l'étudiant change son mot de passe (saisie du mot de passe actuel, du nouveau, puis de sa confirmation). Il peut ensuite consulter et modifier ses coordonnées et sa photo de profil ; toute modification est immédiatement visible dans l'espace d'administration.



Figure 20 : Interface « Mon profil »

Dans l'onglet « Formations », l'étudiant consulte la liste des formations ouvertes avec le nombre de places disponibles — seules les formations ouvertes par la direction sont affichées. Il sélectionne sa spécialité, clique sur « S'inscrire » et confirme dans la boîte de dialogue. L'inscription est enregistrée instantanément : la section « Formation » affiche désormais sa spécialité, et un paiement d'inscription est automatiquement créé avec le statut « En attente ».

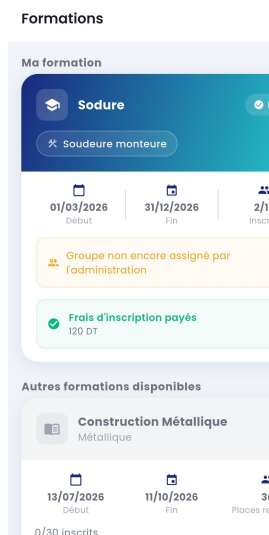


Figure 21 : Inscription à une formation côté étudiant

Une fois son paiement d'inscription validé par l'administration, l'étudiant ouvre l'onglet « Hébergement ». L'application vérifie automatiquement ses critères d'éligibilité, à savoir une inscription active, un paiement confirmé et une distance supérieure à trente kilomètres, et une bannière verte confirme l'éligibilité lorsque le formulaire devient accessible. L'étudiant choisit le type d'hébergement (par exemple « Pension complète », qui inclut les repas au centre), sélectionne ses dates de séjour, indique son motif puis soumet la demande. Celle-ci est enregistrée dans Firestore, peut être téléchargée au format PDF, et une bannière orange indique qu'elle est en cours de traitement.

Dès que l'administrateur approuve la demande, et sans aucune action de l'étudiant, l'application mobile reçoit les changements via les flux Firestore : le badge de statut passe de « Non inscrit » à « Résident », la carte de la chambre attribuée apparaît avec tous les détails (numéro, bloc, période) et la demande porte le badge « Approuvé » en vert. La section « Mes binômes » affiche en temps réel les autres résidents de la même chambre, chacun représenté par un avatar à initiales et son nom complet ; si l'administrateur affecte un nouveau résident à la même chambre, celui-ci apparaît automatiquement, sans rechargement.



Figure 22 : Chambre attribuée et section « Mes binômes »

Dans l'onglet « Paiements », l'étudiant consulte l'ensemble de ses transactions : le paiement d'inscription marqué « Payé » et le paiement d'hébergement en attente, chaque entrée affichant le montant, la date, le type et la référence. Pour chaque paiement validé, il génère et télécharge un reçu officiel au format PDF directement depuis son téléphone ; le document contient l'en-tête du centre, l'identité de l'étudiant, le détail du paiement et la référence de la transaction. Enfin, chaque acteur de la plateforme peut se déconnecter en cliquant sur le bouton « Déconnexion », après une demande de confirmation.



Figure 23 : Historique des paiements et génération du reçu PDF

5.6 Génération de documents PDF

La génération de documents, qu'il s'agisse des fiches étudiants, des reçus de paiement ou des attestations d'hébergement, s'appuie sur les packages pdf et printing, qui permettent de construire des documents structurés côté application, puis de les prévisualiser, de les imprimer ou de les partager directement depuis l'interface, sans dépendance à un service externe de génération de documents. Cette approche garantit un rendu homogène (en-tête du centre, identité de l'étudiant, détails de la transaction) quel que soit le support, web ou mobile, depuis lequel le document est généré.

5.7 Sécurité des données

L'accès aux données est protégé à deux niveaux : l'authentification Firebase, qui garantit que seul un utilisateur connecté peut accéder à l'application, et les règles de sécurité Firestore, qui contrôlent finement, côté serveur, les droits de lecture et d'écriture selon le rôle de l'utilisateur et la correspondance entre l'identifiant du document consulté et celui de l'utilisateur authentifié. Ces règles s'appliquent indépendamment de l'interface utilisée : même un client falsifié ne pourrait contourner les contrôles, puisqu'ils sont évalués par l'infrastructure Firebase elle-même et non par le code de l'application.

5.8 Conclusion

Ce chapitre a illustré, à travers les interfaces développées, l'ensemble des fonctionnalités de la plateforme CFSCMS, du côté administrateur comme du côté étudiant, en insistant sur la dimension temps réel qui en constitue la valeur ajoutée majeure. Cette réalisation concrétise l'ensemble de la démarche d'analyse et de conception présentée dans les chapitres précédents.

Conclusion générale et perspectives

Bilan du projet

Ce travail s'inscrit dans le cadre de la réalisation d'un stage d'été, dans le but de concevoir et d'implémenter « CFSCMS », une plateforme web et mobile de gestion des hébergements étudiants du Centre Sectoriel de Formation en Construction Métallique et Soudure de Médénine. En premier lieu, une étude préliminaire a permis de présenter l'organisme d'accueil et le cadre général du projet, d'identifier les acteurs, d'étudier et de critiquer les solutions existantes, puis de dégager la problématique et les objectifs à atteindre. En deuxième lieu, l'analyse des besoins a permis de spécifier les besoins fonctionnels et non fonctionnels ainsi que les règles de gestion de la plateforme. En troisième lieu, la conception a permis de modéliser le système à travers les diagrammes de cas d'utilisation, de classes, de séquence, ainsi que le modèle de données Firestore. En quatrième lieu, l'environnement de travail ainsi que les choix technologiques retenus, à savoir Flutter, Dart et l'écosystème Firebase, ont été présentés et justifiés. Enfin, la réalisation a permis de développer l'ensemble des modules de la plateforme, illustrés à travers les interfaces développées côté administrateur et côté étudiant.

La plateforme réalisée couvre l'intégralité du cycle administratif de l'étudiant : création du compte avec matricule automatique, inscription aux formations, vérification automatique de l'éligibilité à l'hébergement, attribution des chambres par transactions atomiques, suivi des paiements avec reçus PDF officiels et notifications en temps réel. L'architecture retenue, combinant Flutter pour le développement multiplateforme et Firebase comme backend temps réel, a permis de répondre efficacement aux exigences de cohérence, de sécurité et de disponibilité identifiées lors de l'analyse des besoins. Elle apporte au centre un gain de temps considérable, une fiabilité accrue des données et une visibilité permanente, tout en offrant à l'étudiant une autonomie complète depuis son smartphone.

Sur le plan personnel, ce projet a été l'occasion de mettre en pratique l'ensemble du cycle de développement d'une application, de l'analyse des besoins jusqu'au déploiement en production, en passant par la modélisation d'un domaine métier réel, la conception d'une architecture logicielle en couches, l'implémentation de règles métier transactionnelles

complexes, et la sécurisation d'une base de données partagée entre plusieurs profils d'utilisateurs.

Perspectives d'évolution

Ce travail peut être enrichi par plusieurs perspectives d'évolution, parmi lesquelles :

- l'intégration d'un module de paiement en ligne, permettant à l'étudiant de régler directement ses échéances depuis l'application mobile ;
- la mise en place de notifications push (Firebase Cloud Messaging) pour alerter l'étudiant même lorsque l'application est fermée, sans nécessiter son ouverture ;
- l'ajout d'un mode de fonctionnement hors-ligne avec synchronisation différée, pour les zones à connectivité limitée ;
- la mise en place de tests automatisés (unitaires et d'intégration) et d'un pipeline d'intégration continue, afin de fiabiliser les évolutions futures ;
- l'enrichissement du tableau de bord avec des statistiques avancées et des exports périodiques (Excel, graphiques d'occupation dans le temps) à destination de la direction ;
- la publication de l'application sur le Google Play Store et l'étude d'un support de la plateforme iOS ;
- le support multilingue (arabe, français, anglais), pour une meilleure accessibilité auprès de l'ensemble des stagiaires ;
- un module de gestion de la restauration, lié aux formules de pension choisies par les résidents ;
- l'extension éventuelle de la plateforme à d'autres centres de formation professionnelle, avec une gestion multi-centres.

Nétographie

Les ressources suivantes ont été consultées au cours de la réalisation de ce projet (2026) :

- [1] Flutter — documentation officielle : <https://docs.flutter.dev/>
- [2] Dart — documentation officielle : <https://dart.dev/>
- [3] Firebase — documentation officielle : <https://firebase.google.com/docs>
- [4] Cloud Firestore — modèle de données et règles de sécurité : <https://firebase.google.com/docs/firestore>
- [5] Firebase Authentication : <https://firebase.google.com/docs/auth>
- [6] Firebase Hosting : <https://firebase.google.com/docs/hosting>
- [7] Provider — gestion d'état Flutter : <https://pub.dev/packages/provider>
- [8] Bibliothèques pdf et printing pour Flutter : <https://pub.dev/packages/pdf>
- [9] Documentation des packages Dart : <https://pub.dev>
- [10] Material Design — lignes directrices : <https://m3.material.io>
- [11] Diagrammes UML — cas d'utilisation : <https://www.ibm.com/docs/fr/rational-soft-arch>
- [12] Draw.io — outil de diagrammation : <https://www.drawio.com/>
- [13] Visual Studio Code : <https://code.visualstudio.com/>